TypeScript_5_6

必須パラメータ

javascriptには引数の数に関係なく関数を呼び出すことができる。

```
function my(name, age) {
    console.log(`${name} / ${age}`);
}

my("a", 1, "b");
```

```
a / 1
```

```
function my(name: string, age: number) {
    console.log(`${name} / ${age}`);
}

my("a", 1, "b");
```

Errors in code

```
Expected 2 arguments, but got 3.
```

しかし、TypeScriptは必須パラメータが存在する。

選択的なパラメータ

```
function my(name: string, age?: number) {
    console.log(`${name} / ${age}`);

    if(age){
        console.log(`number : ${age}`);
    }
}

my("yun");
my("yun", undefined);
my("yun",29);
```

```
[LOG]: "yun / undefined"
[LOG]: "yun / undefined"
[LOG]: "yun / 29"
[LOG]: "number : 29"
```

このように引数に?を付けて意図的にundefinedを使用することができる。

```typescript
function my(name: string, age = 29) {
    console.log(`${name} / ${age}`);


}

my("yun");
my("yun", undefined);
my("yun",29);
```

```
[LOG]: "yun / 29"
[LOG]: "yun / 29"
[LOG]: "yun / 29"
```

このよう引数にに値をを付けることもできる。

関数タイプ

javascriptは関数を値で渡すことができる。

```typescript
const songs = ["Juice", "Shake It Off", "What's Up"];

function runOnSongs(getSongAt: (index: number) => string) {
  for (let i = 0; i < songs.length; i += 1) {
    console.log(getSongAt(i));
  }
}

function getSongAt(index: number) {
  return `${songs[index]}`;
}

runOnSongs(getSongAt);

function logSong(song: string) {
  return `${song}`;
}


runOnSongs(logSong);
```

```
[LOG]: "Juice"
[LOG]: "Shake It Off"
[LOG]: "What's Up"
[LOG]: "0"
[LOG]: "1"
[LOG]: "2"
```

Errors in code

```
Argument of type '(song: string) =>
string' is not assignable to
parameter of type '(index: number) =>
string'.
  Types of parameters 'song' and
'index' are incompatible.
    Type 'number' is not assignable
to type 'string'.
```

関数タイプはコールバック関数を説明する時ににによく使われる。

例えば、runOnSongs関数は、引数 getSongAtのタイプをnumber を受け取って string を返す関数として宣言している。

getSongAtを渡すと、タイプが一致するので問題がない。

しかし、logSongは引数にnumberではなくstringを使うため、値を正しく受け取ることができず、エラーになる。

Returnタイプ

```
let songLogger: (song: string) => void;

songLogger = (song) => {
  console.log(`${song}`);
};

songLogger("애국가");
```

```
[LOG]: "애국가"
```

javascriptはReturn値がない場合は基本的にundefinedになる。

しかし、voidを使ってReturnタイプを無視することができる。

```typescript
function fail(message: string): never {
  throw new Error(`Invariant failure: ${message}.`);
}

function workWithUnsafeParam(param: unknown) {
  if (typeof param !== "string") {
    fail(`param should be a string, not ${typeof param}`);
  }

  param.toUpperCase();
}
```

neverタイプは意図的にエラーを発生することのように返すことをしないReturnタイプである。

配列

```typescript
let array: string | number[];

const arrayTwo = ["a", 1, "c", 4];
```

UNION配列はタイプが以上のはいれつである。

```typescript
let arr = [];    // タイプは any

arr.push(' ');  // タイプは  string

arr[0] = 0;     // タイプは  number|string
```

配列にタイプを付けない場合はanyになる。

そして、このようにタイプが変更される場合もある。

スプレッド

```
const existingLogs: string[] = ["Login_Success", "Page_View"];
const newLogs: string[] = ["Button_Click", "Logout"];

const totalHistory: string[] = ["Session_Start", ...existingLogs, ...newLogs];

console.log(totalHistory);
```

```
[LOG]: ["Session_Start",
"Login_Success", "Page_View",
"Button_Click", "Logout"]
```

TypeScriptに**スプレッド**は...を使って配列をそれぞれに示すことである。

このように...をつけて配列を表示することができる。